



Abschlussprüfung Sommer 2026

Fachinformatiker für Anwendungsentwicklung
Dokumentation zur betrieblichen Projektarbeit

Schlüsselinventur

Neuentwicklung dynamischer Inventur-Berichte für die
Schlüsselexemplar-Inventur im System agentes Key Store
Management

Abgabetermin

09. Juni 2026

Prüfungsbewerber

Mousa Al Ataallah



Ausbildungsbetrieb
agentes solutions GmbH

Inhaltsverzeichnis

Inhaltsverzeichnis	II
Abbildungsverzeichnis	IV
Tabellenverzeichnis	IV
Abkürzungsverzeichnis	V
1 Ausgangssituation und Zielsetzung	7
1.1 Projektumfeld und Auftraggeber.....	7
1.2 Projektziel	7
1.3 IST-Zustand und Problemstellung	7
1.3.1 IST-Zustand	7
1.3.2 Problemstellung	8
1.3.3 Auswirkungen der aktuellen Situation	8
1.4 SOLL-Konzept	8
1.5 Projektbeteiligte, Schnittstellen und Abgrenzung	8
2 Projektplanung	9
2.1 Vorgehensmodell und Projektphasen	9
2.2 Zeit- und Ablaufplanung	10
2.3 Ressourcenplanung	11
2.3.1 Personalplanung	11
2.3.2 Sachmittelplanung	11
2.4 Kostenplanung	11
2.5 Wirtschaftlichkeitsbetrachtung	12
3 Durchführung und Realisierung	13
3.1 Architektur und Datenmodell	13
3.1.1 Datenmodell und fachliche Struktur	13
3.1.2 Datenbereitstellung durch Views und Groovy	13
3.2 Prozessschritte der Implementierung	13
3.2.1 Überarbeitung der Inventurliste (Laufzettel)	14
3.2.2 Dynamische Steuerung und Differenzliste	14
3.2.3 Datenaufbereitung und Systemintegration	14
3.3 Entscheidungen und Planabweichungen	15
3.3.1 Anpassung der Zeit- und Phasenplanung	15
3.3.2 Technologische Entscheidungen	15
3.3.3 Fachliche und konzeptionelle Präzisierungen	16
	II

3.3.4	Optimierung des Layouts durch Verzicht auf die Kommentarspalte	16
4	Projektergebnisse und Reflexion	17
4.1	Qualitätssicherung und Testphase	17
4.1.1	Qualitätsanforderungen	17
4.1.2	Testdurchführung	18
4.2	Soll-Ist-Vergleich	19
4.2.1	Fachlicher Soll-Ist-Vergleich	19
4.2.2	Zeitlicher Soll-Ist-Vergleich	19
4.3	Projektabschluss und Übergabe	20
4.4	Fazit und gewonnene Erkenntnisse	21
5	Literaturverzeichnis	22
6	Anhänge	23
6.1	Planungsunterlagen	23
6.2	Kunden-/Anwenderdokumentation	26
6.2.1	Zweck der Funktion	26
6.2.2	Systemvoraussetzungen	26
6.2.3	Voraussetzungen und Berechtigungen	26
6.2.4	Aufruf der Inventur	26
6.2.5	Bearbeitung der Inventur	27
6.2.6	Ausgabe der Berichte	28
6.2.7	Beschreibung der Inventurliste	28
6.2.8	Beschreibung der Inventurdifferenzen.....	30
6.2.9	Hinweise zur Nutzung	31
6.3	Code-Ausschnitte	32
6.3.1	Views	32
6.3.2	Inventurliste	33
6.3.3	Inventurdifferenz	35
6.3.4	Inventurdifferenzen.....	37
7	Eidesstattliche Erklärung	38

Abbildungsverzeichnis

Abbildung 1: EERM-Datenmodell der Inventur-Logik	23
Abbildung 2: Zeitplanung als Gantt-Diagramm	23
Abbildung 3: Layout-Grobentwurf (Mockup) der Inventurliste	24
Abbildung 4: Layout-Grobentwurf (Mockup) der Inventurdifferenzliste	25
Abbildung 5: Layout-Grobentwurf (Mockup) der Inventurdifferenzen.....	25
Abbildung 6: Inventurübersicht mit Auswahl einer Inventur	27
Abbildung 7: Bearbeitungsansicht einer Inventur mit Einträgen und Aktionsmenü ...	28
Abbildung 8: Beispiel einer erzeugten Inventurliste	29
Abbildung 9: Zusätzliche Seite zur manuellen Erfassung	29
Abbildung 10: Beispiel eines Berichts „Inventurdifferenz“	30
Abbildung 11: Gesamtübersicht der Inventurdifferenzen	31
Abbildung 12: Angepasste Inventur-Report-View für die Berichtsausgabe	32
Abbildung 13: Erweiterung der Schrank-View um die Gebäudedaten	32
Abbildung 14: Inventurliste – Root-Skript zur Kategorie- und Report-Steuerung	33
Abbildung 15: Exemplare-Skript zur Schlüsselbund-Auflösung	33
Abbildung 16: Dynamische Spaltensteuerung nach Kategorie im JRXML	34
Abbildung 17: Gruppierungslogik nach Inventurumfang und Kategorie	34
Abbildung 18: Zusätzliche Seite zur Notierung gefundener Schlüssel	35
Abbildung 19: Groovy-Skript zur Filterung nicht bestätigter Einträge	35
Abbildung 20: Filterung der Inventureinträge für die Inventurdifferenzliste	36
Abbildung 21: Aufbereitung von Schlüsselbund- und Untereinträgen	36
Abbildung 22: Erweiterung der Objekt-View um Schrank- und Gebäudedaten	37

Tabellenverzeichnis

Tabelle 1: Projektphasen & Zeitplanung	10
Tabelle 2: Personalplanung	11
Tabelle 3: Sachmittelplanung	11
Tabelle 4: Kostenplanung	12
Tabelle 5: Qualitätsanforderungen	17
Tabelle 6: Testprotokoll der Berichtsfunktionen	18
Tabelle 7: Vergleich der fachlichen Anforderungen	19
Tabelle 8: Vergleich der geplanten und tatsächlichen Projektzeiten	20

Abkürzungsverzeichnis

Abkürzung	Bedeutung
aKSM	agentes Key Store Management
EERM	Erweitertes Entity-Relationship-Modell
GmbH	Gesellschaft mit beschränkter Haftung
GUI	Graphical User Interface (Grafische Benutzeroberfläche)
IDE	Integrated Development Environment (Integrierte Entwicklungsumgebung)
JDK	Java Development Kit (Java-Entwicklungswerkzeuge)
JPQL	Java Persistence Query Language
JRXML	JasperReports XML (Dateiformat für die Berichts-Layouts)
KRITIS	Kritische Infrastrukturen
MySQL	Relationales Datenbankmanagementsystem
n:1	Kardinalität (Beziehungstyp: viele zu eins)
PDF	Portable Document Format
SQL	Structured Query Language (Standard-Abfragesprache für Datenbanken)
XML	Extensible Markup Language (Erweiterbare Auszeichnungssprache)

1 Ausgangssituation und Zielsetzung

1.1 Projektumfeld und Auftraggeber

Die agentes solutions GmbH am Standort Stuttgart ist Teil der bundesweit agierenden agentes Unternehmensgruppe, einem mittelständischen IT-Dienstleister mit weiteren Niederlassungen in München, Kassel und Aschaffenburg. Innerhalb der agentes solutions GmbH entwickeln insgesamt etwa 40 Mitarbeitende innovative Softwarelösungen zur Automatisierung und Digitalisierung von Geschäftsprozessen, vor allem für Unternehmen aus dem Finanzdienstleistungssektor.

Das Projekt findet im Bereich der Produktentwicklung für das System agentes Key Store Management (aKSM) statt.

aKSM ist eine webbasierte Anwendung, die eine professionelle, revisionssichere und effiziente Verwaltung von physischen Schlüsselbeständen ermöglicht.

Der Projektauftrag kommt intern vom Produktmanagement der agentes solutions GmbH. Die Anforderungen basieren auf den Wünschen von mehreren Kunden und wurden direkt in das Fachkonzept der aktuellen Version aufgenommen. Während ich die Details und benötigten Datenfelder in der Analysephase mit der Fachabteilung abstimme, begleitet mein Betreuer die technische Umsetzung.

1.2 Projektziel

Das Ziel meines Projekts ist es, die Berichtsfunktionen für die Inventur in aKSM technologisch neu zu entwickeln und auf das modernisierte n:1-Datenmodell zur eindeutigen Erfassung einzelner Objektexemplare umzustellen. Im Rahmen des Projekts werden hierfür zwei zentrale Berichte neu konzipiert: ein Laufzettel für die physische Prüfung vor Ort sowie eine Differenzliste für die anschließende fachliche Auswertung.

Kern des Projekts ist die Implementierung einer Logik zur automatischen Auflösung von Schlüsselbunden in ihre jeweiligen Einzelschlüssel. Zudem sollen wichtige Identifizierungsdetails wie Gravurnummern über eine gezielte Datenaufbereitung aus bestehenden Datenbank-Views in die Berichtsstrukturen eingebunden werden.

Am Ende sollen die Kunden eine revisionssichere PDF-Lösung erhalten, die den manuellen Aufwand bei der Bestandsprüfung deutlich reduziert und eine lückenlose Dokumentation für jedes Teilobjekt sicherstellt.

1.3 IST-Zustand und Problemstellung

1.3.1 IST-Zustand

Unsere Kunden führen mit dem System aKSM regelmäßig Inventuren durch, um den systemseitigen Buchbestand mit den physischen Schlüsseln vor Ort abzugleichen. Technisch wurde die Datenbank bereits auf ein modernisiertes n:1-Modell umgestellt, welches die Einzel-Erfassung jedes Exemplars ermöglicht. Die PDF-Listen für die Prüfung nutzen jedoch noch eine veraltete Berichtslogik, die auf einer Aggregation

auf Objektebene basiert. Dadurch können die neuen, auf das Exemplar bezogenen, Details in den aktuellen Listen bisher nicht abgebildet werden.

1.3.2 Problemstellung

Die aktuelle Berichtsstruktur führt dazu, dass ein Schlüsselbund auf der Liste lediglich als eine einzige Zeile ausgegeben wird, ohne die darin enthaltenen Einzelschlüssel explizit aufzuführen. Zudem fehlen wesentliche Merkmale, die für eine eindeutige Identifizierung notwendig sind, wie beispielsweise die Gravurnummern oder die Zuordnung zu konkreten Haken- und Lagerplätzen. Dies zwingt die Prüfer dazu, Informationen manuell abzugleichen, die im System bereits digital vorliegen.

1.3.3 Auswirkungen der aktuellen Situation

Die Probleme mit den aktuellen Berichten haben direkte Folgen für den Betrieb:
Wirtschaftlich: Fehler müssen mühsam von Hand im System gesucht werden. Das sorgt laut Fachabteilung für einen Mehraufwand von etwa 15 Minuten pro Inventur.
Sicherheit: Da nicht jeder einzelne Schlüssel erfasst wird, ist die Inventur nicht sicher für offizielle Prüfungen. Ohne Gravurnummern kann man später nicht beweisen, welcher Schlüssel geprüft wurde. Das ist besonders für Kunden aus der kritischen Infrastruktur (KRITIS) ein großes Risiko, weil sie gesetzliche Regeln nicht einhalten können.

1.4 SOLL-Konzept

Das SOLL-Konzept sieht zwei neu entwickelte JasperReports vor: Einen Laufzettel für die Prüfung vor Ort und eine Differenzliste für die Auswertung.

Logik: Wenn ein Schlüsselbund auf der Liste steht, soll das Programm automatisch alle zugehörigen Einzelschlüssel darunter aufzählen.

Datenbeschaffung: Die für die Inventurberichte benötigten Informationen werden über bestehende Datenbank-Views bereitgestellt, die die Daten bereits in einer hierarchisch strukturierten Form aufbereiten.

Die fachspezifische Verarbeitung und Filterung der Datensätze für die Kategorien Depot, Schrank und Person erfolgt dynamisch mittels Groovy-Skripten, einer auf Java basierenden Skriptsprache, die innerhalb von JasperReports zur prozessspezifischen Datenaufbereitung eingesetzt wird. Dadurch wird eine einheitliche Berichtslogik sichergestellt und die technische Komplexität auf Anwendungsebene reduziert.

Flexibilität: Der Bericht reagiert dynamisch auf den jeweiligen Inventurtyp. Je nachdem, ob ein Schrank oder ein Lager geprüft wird, werden nur die passenden Spalten (z. B. „Haken“) angezeigt.

Zusatz: Am Ende des Laufzettels wird eine leere Seite eingefügt, auf der gefundene Schlüssel handschriftlich notiert werden können.

1.5 Projektbeteiligte, Schnittstellen und Abgrenzung

Personelle Schnittstellen

Das Projekt wird durch den Auszubildenden eigenständig umgesetzt. Bei fachlichen

Fragen zu den Anforderungen steht die Fachabteilung (Produktmanagement) als Ansprechpartner zur Verfügung. Eine technische Unterstützung zur Systemarchitektur wird bei Bedarf durch den Ausbilder geleistet.

Technische Umsetzung und Schnittstellen

Die Berichte werden mit JasperReports umgesetzt. Die Datenbereitstellung erfolgt über bestehende Datenbank-Views, während die prozessspezifische Aufbereitung der Daten mithilfe von Groovy-Skripten innerhalb der Reporting-Komponente erfolgt. Die visuelle Gestaltung der Berichte wird in JRXML-Layouts umgesetzt, die in IntelliJ entwickelt und gepflegt werden.

Projektabgrenzung

Um die Eigenleistung klar zu definieren, werden folgende Bereiche explizit vom Projektumfang ausgeschlossen:

Datenbankschema: Es werden keine strukturellen Änderungen am Tabellenaufbau vorgenommen. Das Projekt nutzt das bereits modernisierte n:1-Datenmodell, in dem relevante Attribute wie die „Gravur“ bereits zur Verfügung stehen.

GUI-Entwicklung: Die Erstellung oder Anpassung von grafischen Eingabemasken ist nicht Bestandteil des Projekts. Die Steuerung der Inventuren erfolgt über die bestehende Benutzeroberfläche von aKSM.

Frontend-Logik: Die funktionale Erweiterung der Filtermechanismen im Client liegt außerhalb des Projektfokus. Die Arbeit konzentriert sich ausschließlich auf die Datenbeschaffungsschicht und das Design der PDF-Berichtsstrukturen.

2 Projektplanung

2.1 Vorgehensmodell und Projektphasen

Für die Durchführung des Projekts wird das erweiterte Wasserfallmodell angewendet. Diese Wahl begründet sich durch die fest definierten funktionalen Anforderungen des Fachkonzepts, erlaubt jedoch gleichzeitig notwendige Rückkopplungsschleifen zwischen der Realisierungs- und Entwurfsphase. Dies ist insbesondere für die Gestaltung der Reporting-Komponenten essenziell, um visuelle Details wie Einrückungen in der Darstellung der Schlüsselbunde oder die dynamischen Einblendungen von Spalten iterativ zu optimieren.

Das Projekt gliedert sich in die folgenden vier Hauptphasen:

Analyse und Entwurf: In dieser Phase erfolgt die Einarbeitung in das modernisierte n:1-Datenmodell sowie die technische Analyse der bestehenden Datenbank-Views. Es wird geprüft, in welcher hierarchischen Struktur die Daten vorliegen und wie diese für die Kategorien Depot, Schrank und Person fachlich zugeordnet werden müssen. Ergebnis dieser Phase ist ein technisches Konzept zur Datenanbindung.

Realisierung: Den Schwerpunkt bilden die Datenaufbereitung mittels Groovy-Skripten und das Design der Report-Layouts in JasperReports (JRXML). Hierbei wird die Logik zur automatischen Auflösung von Schlüsselbunden implementiert. Die

Berichte werden so konzipiert, dass sie die hierarchisch strukturierten Daten aus den Views effizient verarbeiten und dynamisch darstellen.

Test und Qualitätssicherung: Nach der Integration der Berichte in das System aKSM wird die korrekte PDF-Generierung anhand verschiedener Testdatensätze, beispielsweise unvollständiger Inventuren oder komplexer Schlüsselbunde, verifiziert. Eventuelle Darstellungsfehler werden in Rückkopplungsschleifen korrigiert.

Abschlussphase: Diese Phase umfasst die Erstellung der Projektdokumentation sowie der Anwenderunterlagen.

2.2 Zeit- und Ablaufplanung

Das Projekt wird im Zeitraum vom 19.05.2026 bis zum 03.06.2026 durchgeführt. Die Planung umfasst insgesamt 80 Arbeitsstunden, die sich auf 10 Arbeitstage mit einer täglichen Arbeitszeit von 8 Stunden verteilen. Die folgende Tabelle schlüsselt die Zeitverteilung auf die Arbeitspakete auf:

Tabelle 1: Projektphasen & Zeitplanung

Projektphase	Teilaufgabe	Dauer
Projektplanung	Projektauftrag prüfen, Vorgehensmodell festlegen, Zeitplan abstimmen	7h
Analysephase & Projektentwurf	IST-Analyse, Anforderungen und technisches Konzept (n:1-Modell)	8h
	Auswertung der hierarchischen View-Strukturen und Entwurf der Layout-Mockups	5h
Implementierung	Datenbeschaffung: Anbindung der Views und Groovy-Datenaufbereitung	7h
	Report 1 (Inventurliste): Implementierung der Logik zur Schlüsselbund-Auflösung	10h
	Report 2 (Differenzliste): Layout-Anpassung sowie Status- und Gesamtlogik	11h
	Integration: Einbindung der Berichte in die Java-Anwendung mithilfe von IntelliJ	9h
Testen & Qualitätssicherung	Erstellung von Testdaten und Durchführung der PDF-Generierungstests	6h
	Fehlerkorrektur und finale Code-Optimierung	4h
Dokumentation & Projektabschluss	Erstellung der Projektdokumentation und Anwenderdokumentation sowie Durchführung des Projektabschlusses	13h
Gesamt		80h

Erläuterung zur Planung: Die Dokumentation wird projektbegleitend sowie in der Abschlussphase erstellt. Der Schwerpunkt der Realisierung begründet sich durch die technische Komplexität der hierarchischen Datenaufbereitung und der Logik zur Schlüsselbund-Auflösung. Die in der Entwurfsphase erstellten visuellen Layouts sind

als Mockups (siehe **Abbildung 3-5**) in Kapitel 3 dokumentiert. Ein detaillierter kalendarischer Ablauf ist als Gantt-Diagramm im Anhang dargestellt (siehe **Abbildung 2**).

2.3 Ressourcenplanung

2.3.1 Personalplanung

Tabelle 2: Personalplanung

Name	Vorname	Funktion	Tätigkeit	Zeitaufwand
Al Ataallah	Mousa	Auszubildender	Projektumsetzung	80 h
		Ausbilder	Fachliche Beratung	nach Bedarf
Fachabteilung	-	Produktmanagement	Anforderungsabstimmung & Abnahme	nach Bedarf

2.3.2 Sachmittelplanung

Die Realisierung erfolgt auf einem Notebook des Unternehmens unter Einhaltung interner IT-Standards. Die für das Projekt eingesetzten Hardware- und Software-Ressourcen sind in der folgenden Tabelle zusammengefasst.

Tabelle 3: Sachmittelplanung

Kategorie	Tool / Ressource	Verwendungszweck
Hardware	Business-Notebook	Intel i7, 16 GB RAM, Windows 11 Pro
Software (IDE)	IntelliJ IDEA	Entwicklungsumgebung
Laufzeitumgebung	Java JDK	Kompilierung und Ausführung der Anwendung
Skripting	Groovy	Prozessspezifische Aufbereitung der Berichtsdaten
Reporting	JasperReports	Design der PDF-Berichte (Open Source)
Versionierung	Git	Quellcode-Management (Open Source)
Datenbank	MySQL	Bereitstellung der Daten über hierarchisch strukturierte Datenbank-Views

2.4 Kostenplanung

Aus datenschutzrechtlichen Gründen werden für die Kalkulation fiktive, branchenübliche Stundensätze verwendet. Da die Umsetzung ausschließlich mit vorhandener Hardware und Open-Source-Software erfolgt, fallen keine zusätzlichen Sachkosten an.

Tabelle 4: Kostenplanung

Position	Zeitaufwand	Stundensatz	Gesamt (netto)
Personalkosten Auszubildender	80 h	55,00 €	4.400,00 €
Personalkosten Fachberatung	4 h	120,00 €	480,00 €
Sachkosten	-	0,00 €	0,00 €
Gesamtkosten des Projekts			4.880,00 €

2.5 Wirtschaftlichkeitsbetrachtung

Die Projektkosten von 4.880,00 € werden dem erwarteten Nutzen gegenübergestellt.

Interner Nutzen (agentes solutions GmbH): Da das Projekt als eigenständiges Lizenzmodul für das System aKSM vertrieben wird, refinanziert sich die Entwicklung bereits durch den Verkauf weniger Lizenzen an Bestands- oder Neukunden. Die Erweiterung steigert zudem die Marktfähigkeit des Produkts im KRITIS-Umfeld.

Externer Nutzen (Kunden): Die automatisierte Datenaufbereitung spart pro Inventur ca. 15 Minuten Bearbeitungszeit. Die Annahme von jährlich 400 Inventuren basiert auf einem repräsentativen Großkunden mit 100 Organisationseinheiten, die jeweils vierteljährlich eine Bestandsprüfung durchführen. Daraus ergibt sich folgende Rechnung:

Jährliche Zeitersparnis: 400 Durchläufe × 15 Min. = 100 Stunden/Jahr.

Jährlicher Kostenvorteil: 100 h × 55,00 € (fiktiv) = 5.500,00 € pro Jahr.

Berechnung der Amortisationsdauer(t): Zur Ermittlung des Zeitpunkts, ab dem die Investition gedeckt ist, wird folgende Formel angewendet:

$$t = \frac{\text{Anschaffungsausgabe}}{\text{durchschnittlicher Rückfluss pro Jahr}}$$

$$t = \frac{4.880,00 \text{ €}}{5.500,00 \text{ €/Jahr}} \approx 0,887 \text{ Jahre}$$

Umrechnung in Monate: 0,887 Jahre × 12 Monate/Jahr ≈ 10,64 Monate.

Die Projektkosten in Höhe von 4.880,00 € amortisieren sich für den Kunden somit nach ca. 11 Monaten.

Qualitative Vorteile:

- **Revisionssicherheit:** Lückenlose Dokumentation durch das neue n:1-Datenmodell.

- **Fehlervermeidung:** Eindeutige Identifizierung durch automatische Schlüsselbund-Auflösung.
- **Zukunftssicherheit:** Flexible Architektur auf Basis von Datenbank-Views und Groovy.

3 Durchführung und Realisierung

3.1 Architektur und Datenmodell

Die technische Basis für die Neuentwicklung der Inventur-Berichte bildet das modernisierte n:1-Datenmodell des Systems aKSM. Im Gegensatz zur früheren Logik, die Bestände nur auf Objektebene abbildete, ermöglicht dieses Modell die eindeutige Erfassung einzelner Objektexemplare. Dies ist die Voraussetzung für eine revisionssichere Darstellung der Inventurergebnisse.

3.1.1 Datenmodell und fachliche Struktur

Das zugrunde liegende Datenmodell bildet die hierarchischen Beziehungen der Schlüsselverwaltung ab (siehe **Abbildung 1**). Die Entitäten Person, Schrank und Depot sind als spezifische Besitzertypen in die bestehende Struktur eingebunden. Jeder InventurEintrag ist mit einem konkreten ObjektExemplar verknüpft, wodurch Merkmale wie Gravurnummern eindeutig ausgewiesen werden können. Die Auflösung von Schlüsselbunden in den Berichten erfolgt über eine Eltern-Kind-Struktur innerhalb der Inventureinträge, welche die hierarchische Zuordnung der Einzelschlüssel technisch definiert.

3.1.2 Datenbereitstellung durch Views und Groovy

Für die Berichtsgenerierung werden die benötigten Daten über vorhandene Datenbank-Views in aufbereiteter Form bereitgestellt. Diese Views bilden komplexe Objektbeziehungen so ab, dass sie in der Reporting-Schicht performant verarbeitet werden können.

Die prozessspezifische Aufbereitung der Daten erfolgt über Groovy-Skripte innerhalb der JasperReports-Komponenten. Hier wird die Logik zur dynamischen Darstellung der Kategorien sowie die visuelle Einrückung der Einzelschlüssel unter einem Bund umgesetzt. Die visuelle Gestaltung und die Definition der Berichtsfelder erfolgen in JRXML-Dateien, die in die Java-Anwendung integriert und in IntelliJ IDEA bearbeitet werden.

3.2 Prozessschritte der Implementierung

Die Realisierung der Berichte wurde in mehreren aufeinander aufbauenden Phasen durchgeführt. Als Orientierung dienten dabei die in der Entwurfsphase erstellten Mockups für die Kategorie Depot sowie für die Inventurdifferenzliste (siehe **Abbildung 3-5**), da diese die grundlegende Layout- und Logikstruktur exemplarisch abbildeten. Zunächst wurde die bisherige Berichtslogik analysiert, um die Unterschiede zwischen der bestehenden Struktur und dem modernisierten n:1-Datenmodell gezielt zu erfassen.

3.2.1 Überarbeitung der Inventurliste (Laufzettel)

Im ersten Schritt wurde die Inventurliste technisch neu konzipiert. Dabei wurden die benötigten Daten aus den vorhandenen Datenbank-Views und aus den Inventur-Einträgen so aufbereitet, dass Merkmale wie die Gravurnummer, der Schrank, der Lagerplatz, die Objektnummer und das jeweilige Exemplar eindeutig ausgewiesen werden können. Dadurch wurde die Grundlage geschaffen, damit die Prüfer vor Ort die Einträge nicht nur vollständig, sondern auch eindeutig nachvollziehen können. Ein Kernpunkt der Implementierung war die Schlüsselbund-Logik. Über eine Eltern-Kind-Zuordnung in den Groovy-Skripten wird automatisiert geprüft, ob ein Eintrag zu einem Bund gehört. In diesem Fall erfolgt eine eingerückte Auflistung der enthaltenen Einzelschlüssel direkt unter dem Bund, wodurch die Struktur des Schlüsselbundes auch im PDF-Bericht nachvollziehbar dargestellt wird. Dies ermöglicht eine lückenlose Kontrolle der einzelnen Teilobjekte und löst das Problem, dass Schlüsselbunde in der bisherigen Berichtlogik nur als Gesamtobjekt dargestellt wurden. Zusätzlich wurde am Ende des Berichts eine statische Seite integriert, die eine handschriftliche Erfassung zusätzlicher Funde erlaubt. Damit wurde eine zentrale fachliche Anforderung aus dem Projektziel direkt in die Berichtslogik übernommen.

3.2.2 Dynamische Steuerung und Differenzliste

Zur Reduzierung des Wartungsaufwands wurde keine getrennte Berichtsdatei für jede Inventurart erstellt, sondern eine generische Berichtsstruktur gewählt. Über einen Parameter im Root-Skript wird die Inventur-Kategorie, also Person, Schrank oder Depot, dynamisch bestimmt und an JasperReports übergeben. In der JRXML-Datei steuert diese Information die Sichtbarkeit der Spalten, wodurch beispielsweise das Feld „Haken“ nur dann eingeblendet wird, wenn es für die jeweilige Inventurart fachlich relevant ist. Durch diese Vorgehensweise kann ein gemeinsamer Bericht für mehrere Anwendungsfälle genutzt werden, was den Pflegeaufwand reduziert und sicherstellt, dass spätere fachliche Änderungen zentral an einer Stelle umgesetzt werden können.

Anschließend wurde die Differenzliste überarbeitet. Hierbei stand die übersichtliche Darstellung aller Abweichungen für den Organisator im Fokus. Zu diesem Zweck wurden zusätzliche Informationen wie das Gebäude, die Objektart, der Lagerort, der Besitzerkontext sowie der jeweilige Entscheidungsstatus, zum Beispiel „Fehlt“ oder „Gefunden“, in die Ausgabe eingebunden. Dadurch dient die Differenzliste unmittelbar als valide Arbeitsgrundlage für die spätere Korrektur der Bestände im System.

3.2.3 Datenaufbereitung und Systemintegration

Die technische Aufbereitung der Daten wurde größtenteils mittels Groovy-Skripting realisiert. Hierbei wurde unter anderem die Sortierlogik definiert, damit die Einträge abhängig vom Inventurumfang nachvollziehbar nach Lagerplatz oder Schrank ausgegeben werden. Zusätzlich wurde eine Zwischenspeicherung geladener Objekte

implementiert, um wiederholte Datenbankzugriffe zu vermeiden und die Performance insbesondere bei größeren Inventuren zu optimieren.

Im letzten Schritt erfolgte die Integration der Berichte in die Java-Anwendung. Hierfür wurden die benötigten Parameter im Root-Skript zusammengeführt und an JasperReports übergeben. Die JRXML-Layouts wurden so finalisiert, dass sie sowohl in der digitalen System-Vorschau innerhalb der aKSM-Benutzeroberfläche als auch im PDF-Druck sauber lesbar sind. Dadurch kann der Anwender die aufbereiteten Inventurdaten bereits vor dem Export prüfen und sicherstellen, dass alle dynamischen Spalten korrekt angezeigt werden. Abschließend wurde die Funktionalität im Rahmen iterativer Entwicklertests mit verschiedenen Testdaten geprüft. Dabei wurde kontrolliert, ob komplexe Schlüsselbunde korrekt aufgelöst und die dynamischen Spalten je nach Kategorie passend ein- oder ausgeblendet werden.

3.3 Entscheidungen und Planabweichungen

Im Verlauf der Umsetzung wurden mehrere technische und konzeptionelle Entscheidungen getroffen, die teilweise von der ursprünglichen Detailplanung aus dem Projektantrag abwichen. Die grundlegenden Projektziele blieben dabei unverändert. Ziel war weiterhin die Neuentwicklung der Inventurliste und der Inventurdifferenzliste auf Basis des modernisierten n:1-Datenmodells sowie die korrekte Verarbeitung von Schlüsselbunden, Exemplaren und zusätzlichen Identifikationsmerkmalen wie Gravurnummern. Die vorgenommenen Anpassungen betrafen daher nicht den fachlichen Projektumfang, sondern die methodische Vorgehensweise und die konkrete technische Realisierung innerhalb der bestehenden Systemarchitektur.

3.3.1 Anpassung der Zeit- und Phasenplanung

Eine wesentliche Planabweichung ergab sich bei der methodischen Strukturierung des Projekts. Im Projektantrag war die Projektplanung nicht als eigenständige Phase mit separatem Zeitblock ausgewiesen. In der tatsächlichen Durchführung erwies sich diese jedoch als notwendig, um Mockups zu erstellen und den technischen Lösungsweg präzise abzustimmen. Dafür wurde ein zusätzlicher Planungsaufwand von 7 Stunden eingeplant.

Diese Verschiebung führte nicht zu einer Überschreitung des Zeitbudgets, da durch die Nutzung vorhandener Datenbank-Views Zeit bei der Realisierung der Datenbeschaffung eingespart wurde. Der geplante Gesamtrahmen von 80 Stunden wurde somit eingehalten.

3.3.2 Technologische Entscheidungen

Eine zentrale technische Entscheidung betraf die Datenbeschaffung. Statt der im Projektantrag genannten JPQL-Neuentwicklung wurden vorhandene Datenbank-Views genutzt. Diese bilden die hierarchischen n:1-Strukturen bereits in geeigneter Form ab und konnten daher effizient in die Berichtserstellung eingebunden werden. Die fachliche Aufbereitung erfolgte stattdessen flexibel in der Reporting-Schicht mittels Groovy-Skripten. Dies reduzierte die Komplexität und verbesserte zugleich die Integration in die bestehende Architektur von aKSM.

Auch die Schlüsselbund-Logik wurde anders umgesetzt als ursprünglich geplant. Anstelle von Sub-Reports wurde die Auflösung der Schlüsselbunde direkt in der Groovy-Datenquelle über eine Eltern-Kind-Struktur realisiert. Diese Variante erwies sich als wartungsärmer, da Sortierung, Einrückung und Datenbelegung zentral gesteuert werden können.

Darüber hinaus wurde statt separater Berichtsdateien für jede Inventurart eine gemeinsame JRXML-Struktur verwendet. Über einen Parameter im Root-Skript wird die jeweilige Kategorie, also Person, Schrank oder Depot, bestimmt. Auf dieser Basis werden die benötigten Spalten im Bericht kontextabhängig ein- oder ausgeblendet. Dadurch konnte die Wartbarkeit der Lösung deutlich verbessert werden.

3.3.3 Fachliche und konzeptionelle Präzisierungen

Im Rahmen der Umsetzung wurde für die Personeninventur zusätzlich ein Besitztyp eingeführt, um zwischen Eigenschlüsseln und Gruppenschlüsseln zu unterscheiden. Dadurch wurde die fachliche Aussagekraft des Berichts verbessert und die tatsächliche Nutzungssituation im Unternehmen präziser abgebildet.

Zudem wurde der Entwurfsumfang bewusst auf Mockups für die Kategorie Depot und die Differenzliste fokussiert, da diese die zugrunde liegende Berichtslogik exemplarisch abbilden. Auf weitere Mockups für jede einzelne Inventurart konnte verzichtet werden, weil die Unterschiede hauptsächlich über die dynamische Spaltensteuerung abgebildet werden.

Zusätzlicher technischer Aufwand entstand außerdem bei der Stabilisierung der Berichtsausgabe. Hierzu wurden Default-Werte für leere Felder ergänzt und eine Zwischenspeicherung geladener Objekte implementiert, um die Performance auch bei größeren Inventuren sicherzustellen.

3.3.4 Optimierung des Layouts durch Verzicht auf die Kommentarspalte

Entgegen der ursprünglichen Planung im Fachkonzept wurde während der Realisierung entschieden, die Spalte „Kommentar“ in den PDF-Berichten der Inventurdifferenzlisten wegzulassen.

Begründung: Durch die Aufnahme zahlreicher neuer Identifikationsmerkmale (Gebäude, Objektart, Schrank, Lagerplatz und Gravurnummern) war der verfügbare horizontale Platz im PDF-Layout vollständig belegt. Eine zusätzliche Kommentarspalte hätte zu einer starken Verkleinerung der Schriftart oder zu unübersichtlichen Zeilenumbrüchen geführt, was die Lesbarkeit und fachgerechte Darstellung erheblich beeinträchtigt hätte. Da Kommentare weiterhin vollständig in der System-GUI von aKSM eingesehen werden können, entsteht für den fachlichen Prozess kein Informationsverlust.

Insgesamt betrafen die Planabweichungen vor allem die konkrete technische Ausgestaltung des Lösungswegs, nicht jedoch die fachliche Zielsetzung des Projekts.

Die getroffenen Entscheidungen führten dazu, dass die Lösung besser in die bestehende Systemlandschaft integriert werden konnte und gleichzeitig wartbar, nachvollziehbar und fachlich nutzbar blieb.

4 Projektergebnisse und Reflexion

4.1 Qualitätssicherung und Testphase

Die Qualitätssicherung wurde im Rahmen einer strukturierten Testphase durchgeführt, die sowohl fachliche als auch funktionale Prüfungen der neu entwickelten Inventurberichte umfasste. Ziel war es, die korrekte Verarbeitung der Daten aus dem neuen n:1-Datenmodell sowie eine fehlerfreie PDF-Erzeugung für unterschiedliche Inventurszenarien sicherzustellen.

4.1.1 Qualitätsanforderungen

Um die Abnahme durch das Produktmanagement und den produktiven Einsatz beim Kunden sicherzustellen, wurden vorab spezifische Qualitätsmerkmale für die Inventur-Berichte definiert. Diese Anforderungen dienen als Maßstab für die anschließende Testphase.

Tabelle 5: Qualitätsanforderungen

Qualitätsmerkmal	Beschreibung/Anforderung
Funktionalität	Die Berichte müssen Daten aus dem n:1-Modell korrekt verarbeiten. Einzelschlüssel müssen lückenlos und eingerückt unter ihrem Schlüsselbund erscheinen.
Statuslogik	Die Differenzliste muss die Status-Werte „Fehlt“ und „Gefunden“ logisch korrekt ausgeben, um eine valide Basis für Bestandsbereinigungen zu bieten.
Vollständigkeit	Alle Inventurarten (Person, Schrank, Depot) müssen vollständig unterstützt werden. Kontextabhängige Daten dürfen dabei nicht fehlen oder falsch zugeordnet sein.
Revisionssicherheit	Jedes Exemplar muss durch die Gravurnummer eindeutig identifizierbar sein, um compliance-relevante Anforderungen insbesondere im KRITIS-Umfeld zu erfüllen.
Benutzbarkeit	Das Layout muss sowohl in der digitalen Systemvorschau als auch im PDF-Ausdruck sauber lesbar und übersichtlich sein.
Effizienz	Die Berichte müssen auch bei großen Beständen ohne merkliche Verzögerung generiert werden.
Änderbarkeit	Die Architektur auf Basis von Datenbank-Views und Groovy-Skripten muss spätere fachliche Anpassungen mit geringem Aufwand ermöglichen.

Im Unterschied zu klassischen Softwareprojekten mit eigenständigen Fachklassen lag der Schwerpunkt nicht auf Unit-Tests, sondern auf der validen Datenaufbereitung durch die Groovy-Skripte und der korrekten visuellen Umsetzung in den JRXML-Layouts. Aufgrund des angewendeten erweiterten Wasserfallmodells erfolgte die Qualitätssicherung iterativ: Festgestellte Darstellungsfehler oder Logiklücken wurden in Rückkopplungsschleifen direkt in der Implementierungsphase behoben.

4.1.2 Testdurchführung

Geprüft wurden im Rahmen der Testläufe insbesondere die korrekte Auflösung von Schlüsselbunden über die Eltern-Kind-Logik, die dynamische Spaltensteuerung sowie die Robustheit der Berichte bei unvollständigen Datensätzen.

Tabelle 6: Testprotokoll der Berichtsfunktionen

Test-ID	Beschreibung	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
T01	Generierung einer Schrankinventur	Bericht wird fehlerfrei erzeugt, Spalte „Haken“ ist sichtbar	Bericht wurde korrekt erzeugt	Erfüllt
T02	Generierung einer Depotinventur	Bericht wird fehlerfrei erzeugt, Spalte „Lagerplatz“ ist sichtbar	Bericht wurde korrekt erzeugt	Erfüllt
T03	Auflösung einer Schlüsselbunds	Eltern-Eintrag mit eingerückten Kind-Einträgen	Hierarchische Darstellung korrekt umgesetzt	Erfolgreich
T04	Personen-Inventur (Eigenschlüssel)	Besitzttyp wird als „Eigener Schlüssel“ ausgewiesen	Besitzttyp korrekt ausgegeben	Erfüllt
T05	Personen-Inventur (Gruppenschlüssel)	Besitzttyp wird als „Gruppenschlüssel“ ausgewiesen	Besitzttyp korrekt ausgegeben	Erfüllt
T06	Prüfung der Identifikationsmerkmale	Gravurnummern werden korrekt angezeigt	Gravurnummer n korrekt zugeordnet	Erfüllt
T07	Robustheitstest mit leeren Feldern	Keine Abbruchfehler, Default-Werte werden verwendet	Stabile PDF-Ausgabe ohne Abweichungen	Erfolgreich
T08	Differenzliste bei Abweichungen	Status-Werte „fehlt“ und „gefunden“ werden korrekt aufgelistet	Differenzen fachlich korrekt dargestellt	Erfolgreich

Die durchgeführten Tests belegen, dass beide Berichte die fachlichen Anforderungen zuverlässig erfüllen.

Nach Abschluss der Korrekturphasen, insbesondere bei der Behandlung optionaler Felder und der Feinjustierung der Einrückungslogik, konnten die Inventurliste und die Inventurdifferenzliste für alle Szenarien fehlerfrei abgenommen werden.

4.2 Soll-Ist-Vergleich

4.2.1 Fachlicher Soll-Ist-Vergleich

Nachfolgend werden die im Soll-Konzept definierten funktionalen Anforderungen dem realisierten Stand gegenübergestellt.

Tabelle 7: Vergleich der fachlichen Anforderungen

Anforderung	Soll-Konzept	Ist-Zustand
Inventurliste	Erzeugung eines dynamischen Laufzettels für die Prüfung vor Ort	Als dynamischer JasperReport umgesetzt.
Differenzliste	Bereitstellung einer übersichtlichen Auswertung der Abweichungen	Vollständig implementiert und auf die Organisatoren-Sicht ausgerichtet. (mit Ausnahme der Kommentarspalte zur Optimierung der Lesbarkeit)
Schlüsselbund-Logik	Auflösung von Bündeln in Einzelschlüssel	Hierarchische Darstellung mit Einrückungen über Groovy umgesetzt
Identifikationsmerkmale	Ausgabe von Gravurnummern zur Identifikation	Integration der Gravurnummern in beide Berichtsformate erfolgt
Dynamische Steuerung	Kontextabhängiges Ein- und Ausblenden von Spalten	Parametergesteuerte Spaltensteuerung implementiert
n:1-Datenmodell	Korrekte Verarbeitung der neuen Exemplar-Strukturen	Datenaufbereitung über Views und Groovy erfolgreich umgesetzt
Inventurarten	Unterstützung der Kategorien Person, Schrank und Depot	Zentrale Steuerung über Parameter im Root-Skript realisiert

4.2.2 Zeitlicher Soll-Ist-Vergleich

Innerhalb der Projektphasen ergaben sich Verschiebungen gegenüber der ursprünglichen Planung. Der fest vorgegebene Gesamtrahmen von 80 Stunden wurde dabei insgesamt eingehalten.

Tabelle 8: Vergleich der geplanten und tatsächlichen Projektzeiten

Projektphase	Soll (h)	Ist (h)	Diff. (h)
Projektplanung	7	7	0
Analyse & Entwurf	13	14	+1
Implementierung	37	35	-2
- Datenbeschaffung	7	5	-2
- Report 1 (Inventurliste)	10	10	0
- Report 2 (Differenzliste)	11	12	+1
- Integration	9	8	-1
Test & Qualitätssicherung	10	11	+1
Dokumentation & Abschluss	13	13	0
Gesamtstunden	80	80	0

Begründung der Abweichungen:

- **Analyse & Entwurf (+1h):** Die komplexe Struktur der vorhandenen Datenbank-Views erforderte eine detailliertere Analyse, um die fachlich korrekte Zuordnung der Felder für alle Kategorien sicherzustellen.
- **Datenbeschaffung (-2h):** Durch die Nutzung vorhandener Datenbank-Views konnte die Datenbereitstellung effizienter als geplant umgesetzt werden.
- **Report 1 (Inventurliste) (0h):** Die Umsetzung erfolgte aufgrund der präzisen Vorbereitung durch Mockups innerhalb des geplanten Zeitrahmens.
- **Report 2 (Differenzliste) (+1h):** Die fachliche Auswertung der Statuswerte sowie die Abbildung der Differenzlogik für den Organisator erforderte einen höheren Implementierungsaufwand.
- **Integration (-1h):** Die Einbindung der Berichte in die Java-Umgebung verlief durch die Nutzung der vorhandenen JasperReports-Struktur effizienter als veranschlagt.
- **Test & Qualitätssicherung (+1h):** Zur Absicherung der Robustheit bei komplexen Schlüsselbund-Konstellationen und unvollständigen Datensätzen wurde eine höhere Anzahl an Testfällen durchgeführt.

4.3 Projektabnahme und Übergabe

Nach erfolgreichem Abschluss der Testphase wurden die Projektergebnisse dem internen Produktmanagement der agentes solutions GmbH sowie dem zuständigen Betreuer zur Abnahme vorgestellt. Im Rahmen eines Abnahmegesprächs am 03.06.2026 wurde die Funktionsfähigkeit der neu entwickelten Inventur-Berichte im System präsentiert.

Besonderes Augenmerk lag dabei auf der korrekten hierarchischen Darstellung der Schlüsselbunde sowie der automatisierten Einblendung der Gravurnummern und

kontextabhängiger Spalten. Die Abnahme erfolgte ohne Beanstandungen, da alle im Soll-Konzept definierten Anforderungen vollständig und fehlerfrei umgesetzt wurden. Im Zuge der Projektübergabe wurden folgende Komponenten bereitgestellt:

- Die finalen **JRXML-Layoutdateien** sowie die zugehörigen **Groovy-Skripte** zur Datenaufbereitung.
- Die **technische Dokumentation** in Form der kommentierten Quellcode-Bestandteile sowie der vorliegenden Projektdokumentation.
- Die **Anwenderdokumentation** (siehe **Anlage 6.2**) zur Einweisung der Fachanwender in die neue Berichtslogik.

4.4 Fazit und gewonnene Erkenntnisse

Das Projekt konnte innerhalb des vorgegebenen Zeitrahmens von 80 Stunden erfolgreich abgeschlossen werden. Durch die technologische Neuentwicklung der Inventur-Berichte auf Basis des modernisierten n:1-Datenmodells wurde eine revisionssichere Lösung geschaffen, die die Anforderungen an die Prüfung vor Ort, die Auswertung von Abweichungen und die organisatorische Gesamtübersicht abdeckt. Dies ist insbesondere für Kunden im KRITIS-Umfeld von hoher Bedeutung, um die lückenlose Dokumentation jedes einzelnen Objektexemplars sicherzustellen. Eine zentrale Erkenntnis des Projekts war die hohe Effizienz der Datenaufbereitung. Die Entscheidung, die Realisierung über vorhandene Datenbank-Views und Groovy-Skripte umzusetzen, erwies sich als architektonisch sinnvoll. Sie reduzierte die Komplexität auf Anwendungsebene und ermöglichte gleichzeitig eine performante Verarbeitung der hierarchischen Strukturen, ohne zusätzliche Abfragen neu entwickeln zu müssen.

Die größte technische Herausforderung bestand in der korrekten Umsetzung der hierarchischen Eltern-Kind-Logik zur Darstellung der Schlüsselbunde. Dies erforderte ein tiefes Verständnis der Datenstrukturen im System aKSM, um eine visuell saubere und fachlich korrekte Einrückung der Einzelschlüssel im PDF-Bericht zu realisieren.

Im Rahmen der Qualitätssicherung wurde festgestellt, dass in den Auswertungsberichten (Inventurdifferenzen) aktuell noch vollständige Detailzeilen mit Checkboxes für jedes Einzelexemplar eines Bundes ausgegeben werden. Da Merkmale wie Besitzer oder Gebäude innerhalb eines Bundes identisch sind und lediglich die Exemplarnummer variiert, besteht hier ein Potenzial zur Reduzierung von Datenredundanz durch eine kompaktere Aggregationslogik.

Insgesamt hat das Projekt gezeigt, dass das erweiterte Wasserfallmodell mit seinen Rückkopplungsschleifen besonders für Reporting-Projekte geeignet ist, da visuelle und fachliche Anpassungen iterativ abgestimmt werden können.

In einem zukünftigen Ausblick wäre eine Erweiterung der Berichte um statistische Auswertungen, wie beispielsweise eine prozentuale Anzeige des Inventurfortschritts, denkbar.

Zudem soll geprüft werden, wie die derzeit aus Platzgründen entfallene Kommentarspalte durch alternative Layout-Optionen oder die zuvor genannte Reduzierung der Datenredundanz wieder integriert werden kann, ohne die Lesbarkeit der Identifikationsmerkmale zu beeinträchtigen.

Damit könnte der Nutzwert des Inventurmoduls für Kunden weiter gesteigert und die Steuerung komplexer Inventurvorgänge verbessert werden.

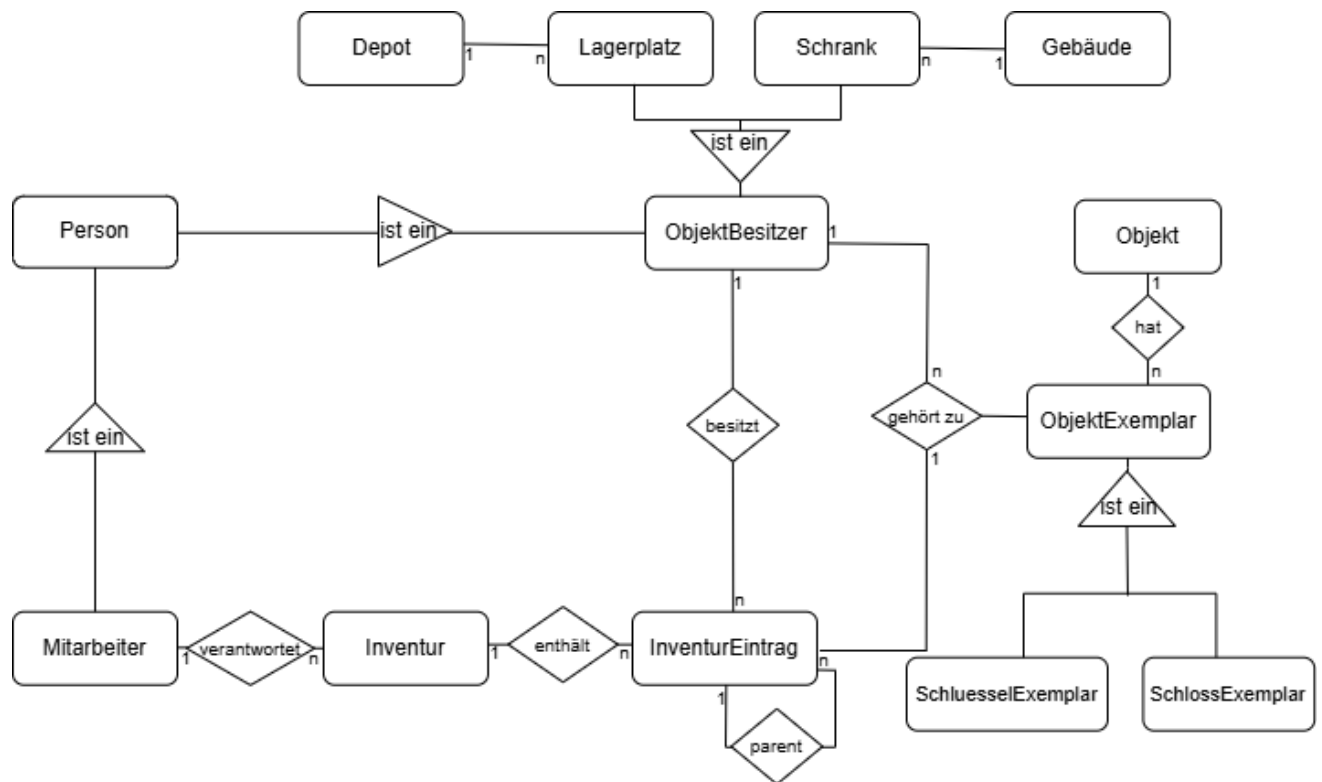
5 Literaturverzeichnis

- Informationen über agentes: <https://www.agentes.de>
- Informationen über aKSM: <https://www.agentes.de/solutions/key-store-manager>
- Fachkonzept (intern): Digitale Schlüsselexemplar-Inventur-Version 6.2.13
- Informationen zu Apache Groovy: <https://groovy-lang.org/documentation.html>
- Informationen zu JasperReports: <https://community.jaspersoft.com>

6 Anhänge

6.1 Planungsunterlagen

Abbildung 1: EERM-Datenmodell der Inventur-Logik



Quelle: Eigene Darstellung unter Verwendung von draw.io

Abbildung 2: Zeitplanung als Gantt-Diagramm

agentes solutions GmbH

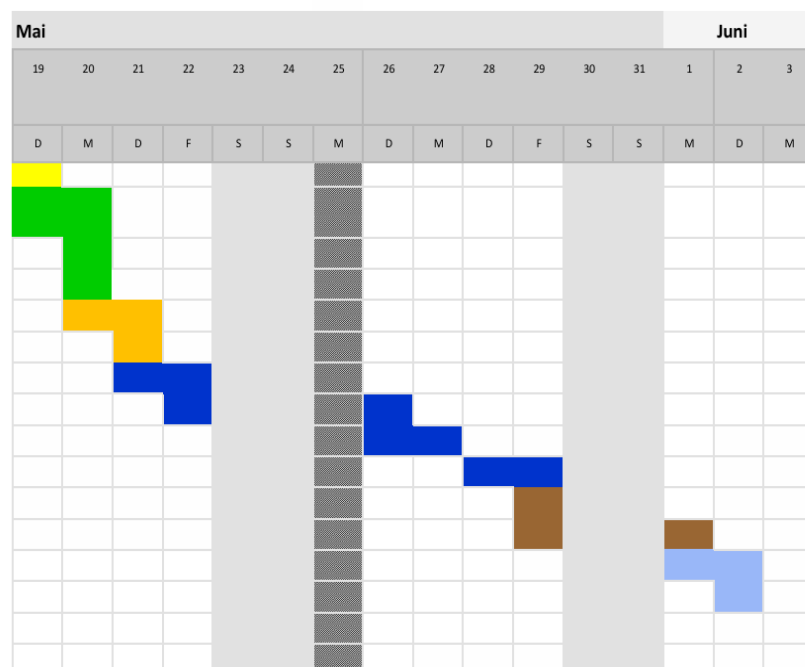
HINWEIS: Am Tag werden nur 8 Stunden gearbeitet

Mousa Al Ataallah

Projektanfangsdatum:

19.05.2026

Meilensteinbeschreibung	Kategorie	Stunden
Projektplanung	Projektplanung	7
Ist-Analyse	Analysephase	5
Anforderungen und technisches Konzept	Analysephase	2
EERM Erstellung	Analysephase	1
Auswertung der View-Strukturen	Projektentwurf	2
Entwurf der Layout-Mockups	Projektentwurf	3
Implementierung der Datenaufbereitung	Implementierung	7
Umsetzung der Inventurliste	Implementierung	10
Umsetzung der Differenzliste	Implementierung	11
Integration der Berichte	Implementierung	9
Testdaten erstellen und PDF-Generierung prüfen	Testen	6
Fehlerkorrektur und finale Optimierung	Testen	4
Projektdokumentation (technisch + fachlich)	Dokumentation	12
Projektabschluss	Dokumentation	1
Gesamt		80



Quelle: Eigene Darstellung unter Verwendung von Excel

Quelle: Die folgenden Mockups werden in draw.io dargestellt.

Abbildung 3: Layout-Grobentwurf (Mockup) der Inventurliste

aKSM - Inventur Depot						
Verantwortlich: Organistator, Olga Fälligkeit				Zeitraum: xx.xx.20xx bis xx.xx.20xx		
Bezeichnung	Nummer	Gravur	Exemplar	Lagerplatz	Bestätigt	Fehlt
ST-Schlüssel	1	G-1111	1	Lagerplatz 1	☐	☐
			2		☐	☐
						Anzahl: 2
Erstellt am xx.xx.2026 xx:xx von Administrator				Seite 1 von 2		

aKSM - Inventur Depot				
Verantwortlich: Organistator, Olga Fälligkeit			Zeitraum: xx.xx.20xx bis xx.xx.20xx	
Bezeichnung	Nummer	Gravur	Exemplar	Kommentar
.....				
.....				
.....				
.....				
.....				
.....				
.....				
.....				
.....				
.....				
.....				
Datum, Unterschrift des Prüfers				
Erstellt am xx.xx.2026 xx:xx von Administrator			Seite 2 von 2	

Abbildung 4: Layout-Grobentwurf (Mockup) der Inventurdifferenzliste

aKSM - Inventurdifferenz Depot									
Verantwortlich: Organistator, Olga					Zeitraum: xx.xx.20xx bis xx.xx.20xx				
Fälligkeit: -									
Bezeichnung	Nummer	Exemplar	Gravur	Lagerplatz	Besitzer	Gebäude	Objektart	Status	Bereinigt
ST-Stuttgart	111		G-1111	Lagerplatz 1	Depot, Lagerplatz1	Stuttgart	Schlüssel	Fehlt	☐
									Anzahl: 1
Erstellt am xx.xx.2026 xxxxx von Administrator									
Seite 1 von 1									

Abbildung 5: Layout-Grobentwurf (Mockup) der Inventurdifferenzen

aKSM - Inventurdifferenzen												
Verantwortlich: Organistator, Olga							Zeitraum: xx.xx.20xx bis xx.xx.20xx					
Fälligkeit												
Bezeichnung	Nummer	Exemplar	Gravur	Objektart	Schrank	Lagerplatz	Gebäude	Besitztyp	Besitzer	Status	Bereinigt	
ST-Schlüssel	2	1	G-1111	Schlüssel	ST-Stuttgart	Lagerplatz1	ST-Gebäude	Lagerplatz	Depot, Lagerplatz1	Fehlt	☐	
											Anzahl: 1	
Erstellt am xx.xx.2026 xxxxx von Administrator												
Seite 1 von 1												

6.2 Kunden-/Anwenderdokumentation

6.2.1 Zweck der Funktion

Die im Rahmen des Projekts neu entwickelten Inventur-Berichte unterstützen die Durchführung und Auswertung von Inventuren im System agentes Key Store Management (aKSM). Ziel ist es, die physische Prüfung von Schlüsseln, Schlüsselbunden und weiteren Objektexemplaren zu erleichtern sowie Abweichungen zwischen Soll- und Ist-Bestand nachvollziehbar darzustellen. Hierfür stehen dem Anwender folgende Berichtstypen zur Verfügung:

- **Inventurliste** zur Durchführung der physischen Prüfung vor Ort,
- **Inventurdifferenzen** zur Auswertung festgestellter Abweichungen.

Die Berichte berücksichtigen das modernisierte n:1-Datenmodell und stellen auch Einzelschlüssel innerhalb eines Schlüsselbundes detailliert dar. Zusätzlich werden je nach Inventurart nur die fachlich relevanten Spalten, wie beispielsweise Lagerplatz, eingeblendet.

6.2.2 Systemvoraussetzungen

Für die Nutzung der Funktion werden folgende Voraussetzungen benötigt:

- Zugriff auf das System aKSM mit einem berechtigten Benutzerkonto.
- ein aktueller Webbrowser zur Nutzung der Anwendung.
- eine PDF-Anzeige zum Öffnen, Speichern oder Drucken der erzeugten Berichte.

6.2.3 Voraussetzungen und Berechtigungen

Vor der Nutzung der Berichte muss eine Inventur im System bereits angelegt oder gestartet worden sein. Die Auswahl und Bearbeitung erfolgt über den Menübereich **Inventuren**.

Die Bearbeitung von Inventuren sowie das Bestätigen oder Markieren von Exemplaren als **Fehlt** ist nur mit den dafür vorgesehenen Rollen und Zuständigkeiten möglich. Bei bestimmten Inventuren kann die Bearbeitung einzelner Exemplare nur durch den jeweils verantwortlichen Benutzer erfolgen.

6.2.4 Aufruf der Inventur

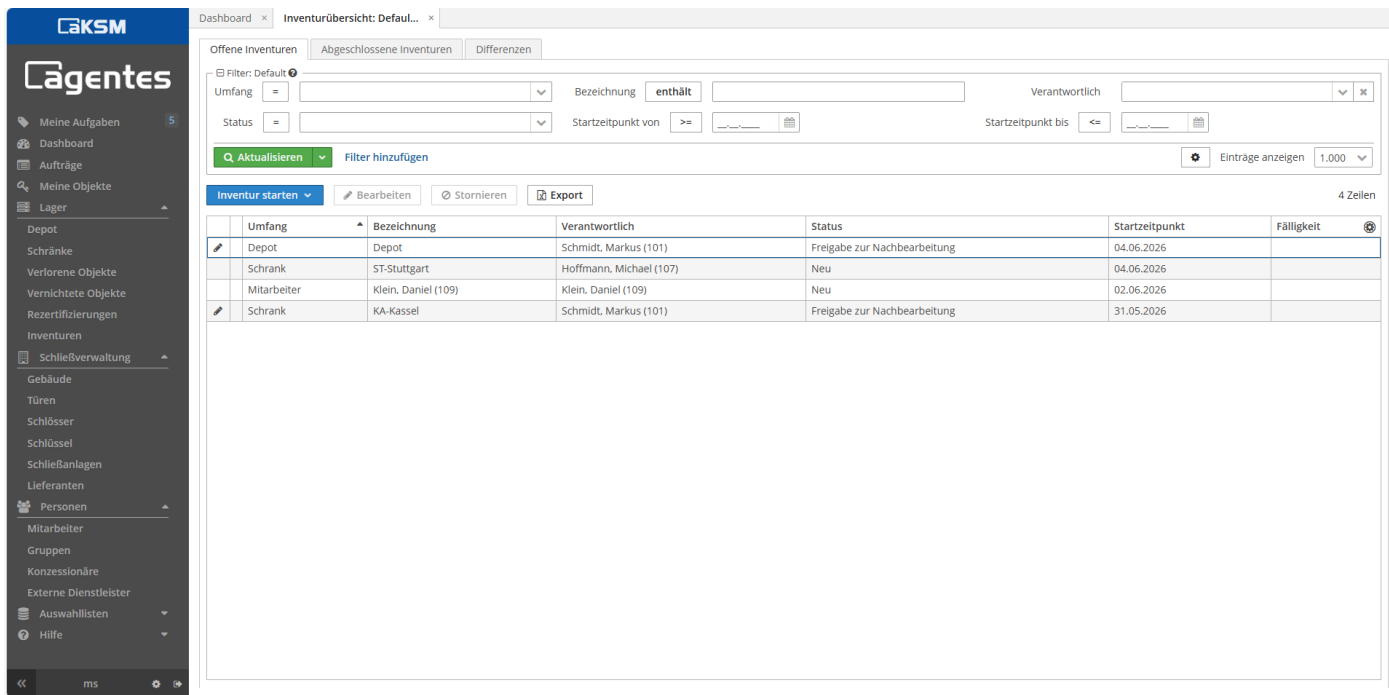
Die Inventurverwaltung wird im linken Navigationsbereich über den Menüpunkt **Inventuren** aufgerufen. Dort können vorhandene Inventuren eingesehen sowie neue Inventuren gestartet werden.

Zum Öffnen einer Inventur sind folgende Schritte erforderlich:

1. Menüpunkt **Inventuren** auswählen.
2. Gewünschte Inventur in der Übersicht markieren.
3. Inventur über **Inventur starten** oder **Bearbeiten** öffnen.

Nach dem Öffnen der Inventur stehen in der Detailansicht die zugehörigen Einträge, Aktionen und Berichtsoptionen zur Verfügung.

Abbildung 6: Inventurübersicht mit Auswahl einer Inventur



The screenshot shows the 'Inventurübersicht' screen in the aKSM application. The interface is divided into a sidebar and a main content area. The sidebar contains navigation links for various functions. The main area has tabs for 'Offene Inventuren', 'Abgeschlossene Inventuren', and 'Differenzen'. Below the tabs are filter options for 'Umfang', 'Bezeichnung', 'Verantwortlich', 'Status', and 'Startzeitpunkt'. A table displays the inventory items with the following data:

Umfang	Bezeichnung	Verantwortlich	Status	Startzeitpunkt	Fälligkeit
Depot	Depot	Schmidt, Markus (101)	Freigabe zur Nachbearbeitung	04.06.2026	
Schränk	ST-Stuttgart	Hoffmann, Michael (107)	Neu	04.06.2026	
Mitarbeiter	Klein, Daniel (109)	Klein, Daniel (109)	Neu	02.06.2026	
Schränk	KA-Kassel	Schmidt, Markus (101)	Freigabe zur Nachbearbeitung	31.05.2026	

Quelle: Eigene Darstellung (System-Screenshot aus aKSM)

6.2.5 Bearbeitung der Inventur

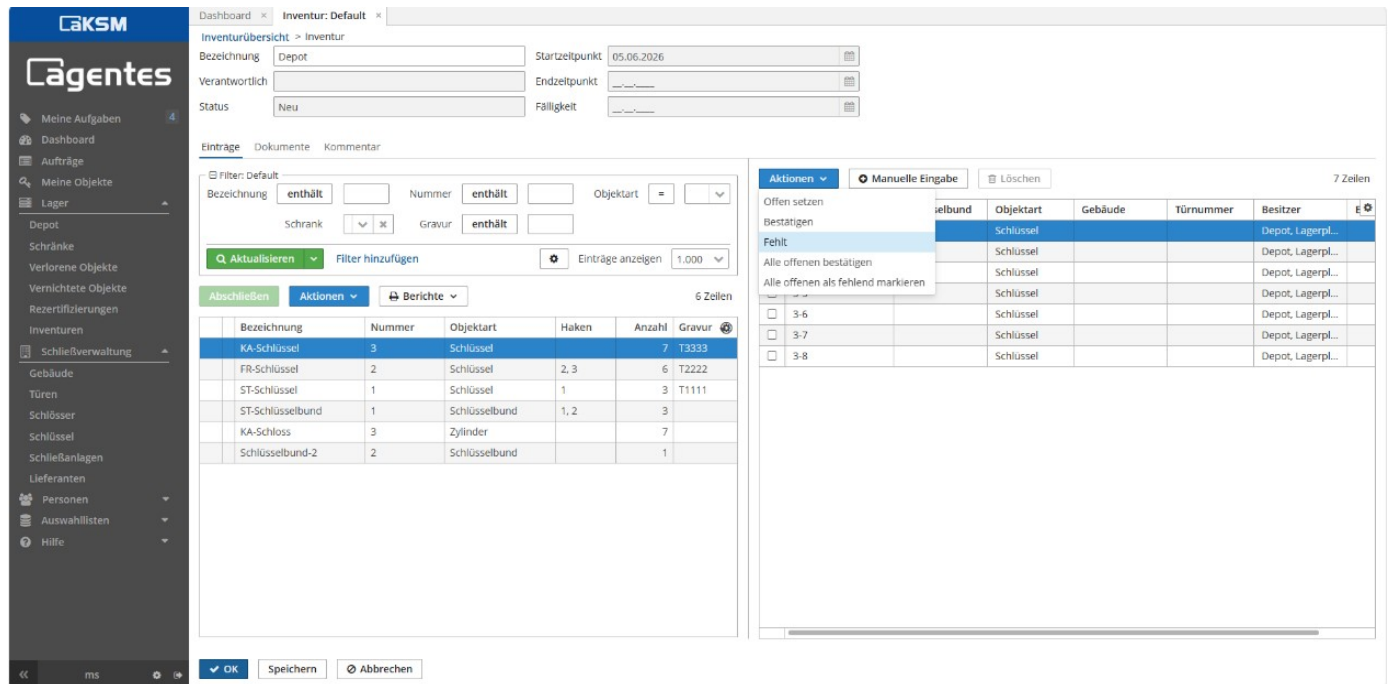
Innerhalb der Inventurbearbeitung werden im Bereich **Einträge** die zur Inventur gehörenden Objekte und Exemplare angezeigt. Abhängig von Rolle und Zuständigkeit können dort verschiedene Aktionen durchgeführt werden.

Zu den möglichen Bearbeitungsschritten gehören insbesondere:

- Exemplare als **Bestätigt** markieren.
- Exemplare als **Fehlt** kennzeichnen.
- offene Einträge gesammelt bearbeiten.
- Inventur nach Abschluss der Prüfung abschließen.

Falls der angemeldete Benutzer nicht über die erforderliche Zuständigkeit verfügt, können bestimmte Aktionen nicht ausgeführt werden. In diesem Fall ist die Bearbeitung durch den fachlich verantwortlichen Benutzer vorzunehmen.

Abbildung 7: Bearbeitungsansicht einer Inventur mit Einträgen und Aktionsmenü



Dashboard **Inventur: Default**

Inventurübersicht > Inventur

Bezeichnung: Depot Startzeitpunkt: 05.06.2026

Verantwortlich: Endzeitpunkt:

Status: Neu Fälligkeit:

Einträge Dokumente Kommentar

Filter: Default

Bezeichnung: enthält Nummer: enthält Objektart: =

Schrank: x Gravur: enthält

Aktualisieren Filter hinzufügen Einträge anzeigen: 1.000

Abschließen **Aktionen** **Berichte** 6 Zeilen

Bezeichnung	Nummer	Objektart	Haken	Anzahl	Gravur
KA-Schlüssel	3	Schlüssel		7	T3333
FR-Schlüssel	2	Schlüssel	2, 3	6	T2222
ST-Schlüssel	1	Schlüssel	1	3	T1111
ST-Schlüsselbund	1	Schlüsselbund	1, 2	3	
KA-Schloss	3	Zylinder		7	
Schlüsselbund-2	2	Schlüsselbund		1	

Aktionen **Manuelle Eingabe** **Löschen** 7 Zeilen

Offen setzen

Bestätigen

Fehlt

Alle offenen bestätigen

Alle offenen als fehlend markieren

3-6

3-7

3-8

OK **Speichern** **Abbrechen**

Quelle: Eigene Darstellung (System-Screenshot aus aKSM)

6.2.6 Ausgabe der Berichte

Die Berichtsausgabe erfolgt innerhalb der Inventurbearbeitung im Bereich **Einträge** über das Dropdown-Menü **Aktionen**. Dort steht der Menüpunkt **Berichte** zur Verfügung.

Über diesen Menüpunkt können die verfügbaren Berichtsarten ausgewählt und als PDF ausgegeben werden:

- **Inventurliste**
- **Inventurdifferenzen**

Die erzeugten Berichte können anschließend gespeichert, geöffnet oder ausgedruckt werden.

6.2.7 Beschreibung der Inventurliste

Die Inventurliste dient als Laufzettel für die physische Prüfung vor Ort. Sie enthält die zu prüfenden Objekte inklusive relevanter Merkmale wie Bezeichnung, Nummer, Gravur und Exemplar.

Schlüsselbunde werden dabei nicht nur als Gesamtobjekt dargestellt, sondern um ihre enthaltenen Einzelschlüssel erweitert. Dadurch kann jedes einzelne Exemplar eindeutig geprüft und dokumentiert werden. Je nach Inventurart werden zusätzliche Informationen wie Lagerplatz eingeblendet. Am Ende der Inventurliste steht eine zusätzliche Seite zur Verfügung, auf der gefundene, bisher nicht erfasste Objekte handschriftlich dokumentiert werden können.

Quelle: Eigene Darstellung (generierter PDF-Bericht aus aKSM)

Abbildung 8: Beispiel einer erzeugten Inventurliste

aKSM - Inventur Depot						
Verantwortlich: -				Start: 07.06.2026		
Fälligkeit: 12.06.2026						
Bezeichnung	Nummer	Gravur	Exemplar	Lager	Bestätigt	Fehlt
ST-Schlüsselbund	 1	-	1	Lagerplatz 1	☐	☐
			2		☐	☐
			5		☐	☐
						Anzahl: 3
ST-Schlüssel	 1	T1111	1	Lagerplatz 1	☐	☐
ST-Schlüssel	 1	T1111	2	Lagerplatz 1	☐	☐
ST-Schlüssel	 1	T1111	3	Lagerplatz 1	☐	☐
			5		☐	☐
			7		☐	☐
			8		☐	☐
						Anzahl: 3
FR-Schlüssel	 2	T2222	1	Lagerplatz 1	☐	☐
			2		☐	☐
			5		☐	☐
			6		☐	☐
			7		☐	☐
			8		☐	☐
						Anzahl: 6

Erstellt am 08.06.2026 16:10 von Schmidt, Markus (101)

Seite 1 von 4

Abbildung 9: Zusätzliche Seite zur manuellen Erfassung

[illegible]

6.2.8 Beschreibung der Inventurdifferenzen

Der Bericht **Inventurdifferenz** dient der fachlichen Auswertung während oder nach der Bearbeitung einer Inventur. Er stellt alle festgestellten Abweichungen übersichtlich dar und unterstützt den Organisator bei der Nachbearbeitung. Hierzu werden insbesondere Statusinformationen wie **Fehlt** sowie weitere Merkmale wie Gravur, Besitzer, Lagerplatz, Gebäude oder Objektart ausgegeben. Dadurch können Unstimmigkeiten gezielt identifiziert und anschließend im System bereinigt werden.

Abbildung 10: Beispiel eines Berichts „Inventurdifferenz“

aKSM - Inventurdifferenz Depot									
Verantwortlich: Schmidt, Markus (101)							Start: 07.06.2026		
Fälligkeit: 12.06.2026									
Bezeichnung	Nummer	Exemplar	Gravur	Lagerplatz	Besitzer	Gebäude	Objektart	Status	Bereinigt
Schlüsselbund-2	2	1		Lagerplatz1	Depot, Lagerplatz1	Kassel	Schlüsselbund	Fehlt	☐
KA-Schlüssel	3	1	T3333	Lagerplatz1	Depot, Lagerplatz1	Kassel	Schlüssel	Fehlt	☐
									Anzahl: 1
KA-Schloss	3	2		Lagerplatz1	Depot, Lagerplatz1	Kassel	Zylinder	Fehlt	☐
KA-Schloss	3	3		Lagerplatz1	Depot, Lagerplatz1	Kassel	Zylinder	Fehlt	☐
KA-Schloss	3	4		Lagerplatz1	Depot, Lagerplatz1	Kassel	Zylinder	Fehlt	☐
KA-Schloss	3	5		Lagerplatz1	Depot, Lagerplatz1	Kassel	Zylinder	Fehlt	☐
KA-Schloss	3	6		Lagerplatz1	Depot, Lagerplatz1	Kassel	Zylinder	Fehlt	☐
KA-Schloss	3	7		Lagerplatz1	Depot, Lagerplatz1	Kassel	Zylinder	Fehlt	☐
KA-Schloss	3	8		Lagerplatz1	Depot, Lagerplatz1	Kassel	Zylinder	Fehlt	☐
Erstellt am 08.06.2026 16:12 von Schmidt, Markus (101)							Seite 1 von 2		

Quelle: Eigene Darstellung (generierter PDF-Bericht aus aKSM)

Gesamtübersicht der Inventurdifferenzen

Die Gesamtübersicht der Inventurdifferenzen zeigt die im System vorhandenen Differenzen in einer zusammenfassenden Ansicht. Sie dient als Arbeitsgrundlage für die fachliche Nachbearbeitung und bietet einen schnellen Überblick über den Status der Inventur.

Abbildung 11: Gesamtübersicht der Inventurdifferenzen

aKSM - Inventurdifferenzen											
Bezeichnung	Nummer	Exemplar	Gravur	Objektart	Schrank	Lagerplatz	Gebäude	Besitztyp	Besitzer	Status	Bereinigt
FR-Schlüssel	2	1	T2222	Schlüssel	FR-Frankfurt	Lagerplatz1	Frankfurt	Lagerplatz	Depot, Lagerplatz1	Fehlt	☐
FR-Schlüssel	2	2	T2222	Schlüssel	FR-Frankfurt	Lagerplatz1	Frankfurt	Lagerplatz	Depot, Lagerplatz1	Fehlt	☐
FR-Schlüssel	2	5	T2222	Schlüssel	FR-Frankfurt	Lagerplatz1	Frankfurt	Lagerplatz	Depot, Lagerplatz1	Fehlt	☐
FR-Schlüssel	2	6	T2222	Schlüssel	FR-Frankfurt	Lagerplatz1	Frankfurt	Lagerplatz	Depot, Lagerplatz1	Fehlt	☐
FR-Schlüssel	2	7	T2222	Schlüssel	FR-Frankfurt	Lagerplatz1	Frankfurt	Lagerplatz	Depot, Lagerplatz1	Fehlt	☐
FR-Schlüssel	2	8	T2222	Schlüssel	FR-Frankfurt	Lagerplatz1	Frankfurt	Lagerplatz	Depot, Lagerplatz1	Fehlt	☐
KA-Schloss	3	2		Zylinder	KA-Kassel	Lagerplatz1	Kassel	Lagerplatz	Depot, Lagerplatz1	Fehlt	☐
KA-Schloss	3	3		Zylinder	KA-Kassel	Lagerplatz1	Kassel	Lagerplatz	Depot, Lagerplatz1	Fehlt	☐
KA-Schloss	3	4		Zylinder	KA-Kassel	Lagerplatz1	Kassel	Lagerplatz	Depot, Lagerplatz1	Fehlt	☐
KA-Schloss	3	5		Zylinder	KA-Kassel	Lagerplatz1	Kassel	Lagerplatz	Depot, Lagerplatz1	Fehlt	☐
KA-Schloss	3	6		Zylinder	KA-Kassel	Lagerplatz1	Kassel	Lagerplatz	Depot, Lagerplatz1	Fehlt	☐
KA-Schloss	3	7		Zylinder	KA-Kassel	Lagerplatz1	Kassel	Lagerplatz	Depot, Lagerplatz1	Fehlt	☐
KA-Schloss	3	8		Zylinder	KA-Kassel	Lagerplatz1	Kassel	Lagerplatz	Depot, Lagerplatz1	Fehlt	☐
KA-Schlüssel	3	9	T3333	Schlüssel	KA-Kassel	Lagerplatz1	Kassel	Schrank	KA-Kassel	Fehlt	☐
KA-Schlüssel	3	10	T3333	Schlüssel	KA-Kassel	Lagerplatz1	Kassel	Schrank	KA-Kassel	Fehlt	☐
ST-Schlüsselbund	1	1		Schlüsselbund	ST-Stuttgart	Lagerplatz1	Stuttgart	Lagerplatz	Depot, Lagerplatz1	Fehlt	☐
ST-Schlüssel	1	3	T1111	Schlüssel						Fehlt	☐
ST-Schlüsselbund	1	5		Schlüsselbund	ST-Stuttgart	Lagerplatz1	Stuttgart	Lagerplatz	Depot, Lagerplatz1	Fehlt	☐
ST-Schlüssel	1	1	T1111	Schlüssel						Fehlt	☐

Erstellt am 06.06.2026 23:39 von Schmidt, Markus (101)

Seite 1 von 2

Quelle: Eigene Darstellung (generierter PDF-Bericht aus aKSM)

6.2.9 Hinweise zur Nutzung

Vor dem Ausdruck sollte geprüft werden, ob die richtige Inventur und die passende Inventurart ausgewählt wurden. Da sich die eingeblendeten Spalten abhängig von Person, Schrank oder Depot unterscheiden können, ist die Darstellung immer kontextbezogen.

Bei fehlenden Bearbeitungsmöglichkeiten ist zu prüfen, ob der angemeldete Benutzer über die erforderlichen Rechte bzw. die fachliche Zuständigkeit verfügt. Die Berichte dienen ausschließlich der Unterstützung des Inventurprozesses. Fachliche Korrekturen der Bestände erfolgen weiterhin im vorgesehenen Arbeitsablauf innerhalb des Systems.

6.3 Code-Ausschnitte

Hinweis: Die dargestellten Codeausschnitte sind im hellen Modus abgebildet, da die Berichtsansicht der Anwendung in dieser Darstellung verwendet wird. Die Groovy-Skripte werden innerhalb der Berichte gepflegt und über IntelliJ sowie Git verwaltet.

6.3.1 Views

Abbildung 12: Angepasste Inventur-Report-View für die Berichtsausgabe

```

846 <view entity="ksm_inventur" name="inventur.report" extends="_local">
847   <property name="eintraege" view="_local">
848     <property name="exemplar" view="_minimal">
849       <property name="objekt" view="_minimal">
850         <property name="schrank" view="_minimal">
851           <property name="gebaeude" view="_minimal"/>
852         </property>
853         <property name="haken" view="_minimal"/>
854         <property name="hakenAnzeige"/>
855         <property name="lagerplatz" view="_local"/>
856         <property name="gravur"/>
857         <property name="kritisch"/>
858       </property>
859       <property name="besitzer" view="_minimal"/>
860     </property>
861     <property name="besitzer" view="_minimal"/>
862     <property name="gebaeude" view="_minimal"/>
863     <property name="parent" view="_minimal">
864       <property name="gebaeude" view="_minimal"/>
865     </property>
866   </property>
867   <property name="verantwortlich" view="_minimal"/>
868   <property name="inventurObjektBesitzer" view="_minimal">
869     <property name="name"/>
870   </property>
871   <property name="depot" view="_minimal">
872     <property name="lagerplaetze" view="_minimal"/>
873   </property>
874 </view>

```

Abbildung 13: Erweiterung der Schrank-View um die Gebäudedaten

```

810 <view entity="ksm_Schrank" name="inventur.schrank" extends="_minimal">
811   <property name="verwalter" view="_minimal"/>
812   <property name="tenant"/>
813   <property name="scope"/>
814   <property name="gebaeude" view="_minimal"/>
815 </view>

```


6.3.2 Inventurliste

Abbildung 14: Inventurliste – Root-Skript zur Kategorie- und Report-Steuerung

Script editor for Inventurliste > Root

```

25
26 def inventur = dataManager.load(Inventur.class)
27   .id(params.get('entity').getId())
28   .view('inventur.report')
29   .one()
30
31 def inventurEntityName = messages.getMessage(Inventur.class, "Inventur")
32 def inventurService = AppBeans.get('ksm_InventurService')
33 def faelligkeitsDatum = inventurService.berechneFaelligkeitsDatum(inventur) ?: '-'
34
35 String reportName = "${inventurEntityName} ${inventur.bezeichnung}"
36
37 String reportFile = "Inventurliste.jrxml"
38 String kategorie
39 if (InventurUmfang.DEPOT.equals(inventur.umfang)) {
40   kategorie = "DEPOT"
41 } else if (inventur.umfang?.isPersonenInventur()) {
42   kategorie = "PERSON"
43 } else {
44   kategorie = "SCHRANK"
45 }
46
47 def alleEintraege = inventur.getEintraege()
48 def regulaereEintraege = alleEintraege.findAll { it.exemplar != null && !InventurEintragEntscheidung.GEFUNDEN.equals(it.entscheidung) }
49
50 def mitHaken = regulaereEintraege
51   .any { !it.exemplar.objekt.haken.isEmpty() }
52
53
54 return [[
55   'agentes.report.file' : reportFile,
56   'kategorie' : kategorie,
57   'userLogin' : user.login,
58   'userName' : user.name,
59   'user' : user,
60   'reportName' : reportName,
61   'gravurErlaubt' : gravurErlaubt,
62   'erstelltAm' : dateTimeFormat.format(timeSource.currentTimestamp()),

```

Hinweis: Der Parameter kategorie wird im Root-Skript der Inventurliste analog zur Inventurdifferenzliste aufgebaut und zur Steuerung des Berichts verwendet. Eine separate Abbildung wurde daher nicht aufgenommen.

Abbildung 15: Exemple-Skript zur Schlüsselbund-Auflösung

Script editor for Inventurliste > exemplar

```

57 // Schlüsselbund wird als eigene Hauptzeile ausgegeben; enthaltene Schlüssel werden zusätzlich als Unterzeilen ergänzt.
58 if (ObjektArt.SCHLUESSELBUND.equals(eintrag.objektArt)) {
59
60   def bundName = eintrag.exemplar.objekt.name
61   // Kind-Einträge des Schlüsselbunds werden einmal ermittelt.
62   def children = inventur.getEintraege().findAll { child ->
63     child.parent != null && child.parent.id == eintrag.id
64   }
65
66   result << [
67     'lagerplatz' : eintrag.exemplar.objekt.lagerplatz.name ?: '-',
68     'schrank' : eintrag.exemplar.objekt.schrank.anzeigeName ?: '-',
69     'haken' : eintrag.exemplar.objekt.hakenAnzeige ?: '-',
70     'gravur' : eintrag.exemplar.objekt.gravur ?: '-',
71     'objectId' : eintrag.exemplar.objekt.id,
72     'name' : eintrag.exemplar.objekt.name ?: '-',
73     'nummer' : eintrag.exemplar.objekt.nummer ?: '-',
74     'exemplar' : eintrag.exemplar.nummerzusatz ?: '-',
75     'besitztyp' : getBesitztyp(eintrag.exemplar.besitzer),
76     'type' : getType(eintrag.objektArt, eintrag.exemplar.objekt.kritisch),
77     'schlüsselbund' : '',
78     'bundParent' : true,
79     'hasChildren' : !children.isEmpty(),
80     'sortierNummer' : eintrag.exemplar.objekt.nummer,
81     'sortierExemplar' : eintrag.exemplar.nummerzusatz
82   ]
83
84   children.each { child ->
85
86     // Das Kind-Objekt wird separat geladen, damit für den enthaltenen Schlüssel alle benötigten Reportdaten vollständig vorliegen.
87     def objekt = loadObjekt(child.exemplar.objekt.id)
88
89     result << [
90       'lagerplatz' : eintrag.exemplar.objekt.lagerplatz.name ?: '-',
91       'schrank' : eintrag.exemplar.objekt.schrank.anzeigeName ?: '-',
92       'haken' : eintrag.exemplar.objekt.hakenAnzeige ?: '-',
93       'gravur' : objekt.gravur ?: '-',
94     ]

```


Abbildung 16: Dynamische Spaltensteuerung nach Kategorie im JRXML

```

115         <staticText>
116             <reportElement x="520" y="23" width="40" height="16" uuid="4ab0b94e-3816-4c8c-a1e3-32b0fac6d112">
117                 <printWhenExpression><![CDATA["SCHRANK".equals($P{kategorie})]]></printWhenExpression>
118             </reportElement>
119             <textElement>
120                 <font size="10" isBold="true"/>
121             </textElement>
122             <text><![CDATA[Haken]]></text>
123         </staticText>
124         <staticText>
125             <reportElement x="520" y="24" width="54" height="16" uuid="4ab0b94e-3816-4c8c-a1e3-32b0fac6d111">
126                 <printWhenExpression><![CDATA["DEPOT".equals($P{kategorie})]]></printWhenExpression>
127             </reportElement>
128             <textElement>
129                 <font size="10" isBold="true"/>
130             </textElement>
131             <text><![CDATA[Lager]]></text>
132         </staticText>
133         <staticText>
134             <reportElement x="520" y="24" width="76" height="16" uuid="4ab0b94e-3816-4c8c-a1e3-32b0fac6d113">
135                 <printWhenExpression><![CDATA["PERSON".equals($P{kategorie})]]></printWhenExpression>
136             </reportElement>
137             <textElement>
138                 <font size="10" isBold="true"/>
139             </textElement>
140             <text><![CDATA[Besitztyp]]></text>
141         </staticText>

```

Hinweis: Der Parameter `kategorie` wird im JRXML der Inventurliste analog zur Inventurdifferenzliste zur dynamischen Spaltensteuerung verwendet

Abbildung 17: Gruppierungslogik nach Inventurumfang und Kategorie

```

51     <group name="GroupUmfang" isStartNewPage="true" isReprintHeaderOnEachPage="true">
52         <groupExpression><![CDATA["DEPOT".equals($P{kategorie}) ? $F{lagerplatz} :
53             "SCHRANK".equals($P{kategorie}) ? $F{schrack} :
54             $P{verantwortlich}]]></groupExpression>

```

Abbildung 18: Zusätzliche Seite zur Notierung gefundener Schlüssel

```

Inventurliste.jrxml x Inventurdifferenz.jrxml Inventurdifferenz.jrxml
333 </band>
334 </pageFooter>
335 <lastPageFooter>
336   <band height="464">
337     <staticText>
338       <reportElement x="0" y="10" width="70" height="16" uuid="ffb0b87a-ce2f-4ea7-b34c-70a30584f5fd"/>
339       <textElement>
340         <font size="10" isBold="true"/>
341       </textElement>
342       <text><![CDATA[Bezeichnung]]></text>
343     </staticText>
344     <staticText>
345       <reportElement x="178" y="10" width="130" height="16" uuid="ee313d2d-670d-4ce5-b57e-4cb46fa56f6d">
346         <printWhenExpression><![CDATA[!$P{gravurErlaubt}]]></printWhenExpression>
347       </reportElement>
348       <textElement>
349         <font size="10" isBold="true"/>
350       </textElement>
351       <text><![CDATA[Nummer]]></text>
352     </staticText>
353     <staticText>
354       <reportElement x="269" y="10" width="130" height="16" uuid="ee313d2d-670d-4ce5-b57e-4cb46fa56f6e">
355         <printWhenExpression><![CDATA[!$P{gravurErlaubt}]]></printWhenExpression>
356       </reportElement>
357       <textElement>
358         <font size="10" isBold="true"/>
359       </textElement>
360       <text><![CDATA[Nummer]]></text>
361     </staticText>
362     <staticText>
363       <reportElement x="393" y="10" width="80" height="16" uuid="a3b4c5d6-1234-4abc-9012-111111111111">
364         <printWhenExpression><![CDATA[!$P{gravurErlaubt}]]></printWhenExpression>
365       </reportElement>
366       <textElement>
367         <font size="10" isBold="true"/>
368       </textElement>
369       <text><![CDATA[Gravur]]></text>
370     </staticText>
371     <staticText>
372       <reportElement x="490" y="10" width="55" height="16" uuid="0d14751a-bad4-4cdf-9ea2-fc4471f5da25"/>
373       <textElement textAlignment="Right">
374         <font size="10" isBold="true"/>
375       </textElement>
376       <text><![CDATA[Exemplar]]></text>
377     </staticText>
378   </band>

```

6.3.3 Inventurdifferenz

Abbildung 19: Groovy-Skript zur Filterung nicht bestätigter Einträge

```

53 def exemplare = alleEintraege
54   .findAll { it.entscheidung &&
55     !InventurEintragEntscheidung.BESTAETIGT.equals(it.entscheidung) }
56

```

Abbildung 20: Filterung der Inventureinträge für die Inventurdifferenzliste

Script editor for Inventurdifferenz > exemplare

Abbildung : Filterung der Inventureinträge für die Inventurdifferenzliste
 Abbildung : Groovy-Skript zur Filterung nicht bestätigter Einträge

```

39 // Falls die Entscheidung, ob InventurEintrag und SchlüsselBUNDESTITELTG übereinstimmt, eine Entscheidung
40 // ist, dann wird die Entscheidung in der Inventurdifferenzliste übernommen.
41 def statusText = messages.getMessage(eintrag.entscheidung)
42 // Schlüsselbunde werden als Hauptzeilen angezeigt, die einzelnen Schlüssel darunter als Detailzeilen
43 if (objektArt.SCHLUESSELBUND.equals(eintrag.objektArt) && eintrag.exemplar != null) {
44   // Lädt das Schlüsselbund-Exemplar mit erweiterter View, damit die enthaltenen Schlüssel für den Bericht verfügbar sind.
45   def exemplar = loadObjekt(eintrag.objekt.id, 'schlüsselbundExemplarAufloesenView')
46   .id(eintrag.exemplar.id)
47   .view('schlüsselbundExemplarAufloesenView')
48   .one()
49   def bundName = eintrag.exemplar.objekt.name
50   // Abgabe der Hauptzeile für den Schlüsselbund in der Differenzliste
51   'schrank' : eintrag.exemplar.objekt.schrank?.anzeigename,
52   'lagerplatz' : eintrag.exemplar.objekt.lagerplatz?.name,
53   'objektid' : eintrag.exemplar.objekt.id,
54   'haken' : eintrag.exemplar.objekt.hakenAnzeige,
55   'gravur' : eintrag.exemplar.objekt.gravur,
56   'name' : bundName,
57   'nummer' : eintrag.exemplar.objekt.nummer,
58   'exemplar' : eintrag.exemplar.nummerzusatz?.toString(),
59   'gebaude' : eintrag.exemplar.objekt.schrank.gebaude?.name?: '',
60   'objektArtText' : eintrag.objektArt?.messages.getMessage(eintrag.objektArt) : '',
61   'kommentar' : eintrag.kommentar?: '',
62   'besitztyp' : eintrag.besitzer?.objektBesitzerTyp?.id?: '',
63   'besitzer' : eintrag.besitzer?.anzeigename?: '',
64   'status' : statusText,
65   'bereinigt' : InventurEintragStatus.BEREINIGT.equals(eintrag.status),
66   'type' : 'SCHLUESSELBUND',
67   'schlüsselbund' : '',
68   'sortierExemplar' : eintrag.exemplar.objekt.sortierExemplar,
69   'sortierNummer' : eintrag.exemplar.objekt.sortierNummer,
70   'sortierZusatz' : eintrag.exemplar.objekt.sortierZusatz
71 }
72

```

Abbildung : Aufbereitung von Schlüsselbund- und Untereinträgen
 Abbildung : Filterung der Inventureinträge für die Inventurdifferenzliste
 Abbildung : Groovy-Skript zur Filterung nicht bestätigter Einträge

Abbildung : Filterung der Inventureinträge für die Inventurdifferenzliste
 Abbildung : Groovy-Skript zur Filterung nicht bestätigter Einträge

Abbildung : Filterung der Inventureinträge für die Inventurdifferenzliste
 Abbildung : Groovy-Skript zur Filterung nicht bestätigter Einträge

Abbildung : Aufbereitung von Schlüsselbund- und Untereinträgen
 Abbildung : Filterung der Inventureinträge für die Inventurdifferenzliste

Abbildung : Aufbereitung von Schlüsselbund- und Untereinträgen

Abbildung : Erweiterung der Objekt-View um Schrank- und Gebäudedaten
 Abbildung : Aufbereitung von Schlüsselbund- und Untereinträgen
 Abbildung : Filterung der Inventureinträge
 Abbildung 21: Aufbereitung von Schlüsselbund- und Untereinträgen

Script editor for Inventurdifferenz > exemplare

Abbildung : Aufbereitung von Schlüsselbund- und Untereinträgen
 Abbildung : Filterung der Inventureinträge für die Inventurdifferenzliste
 Abbildung : Groovy-Skript zur Filterung nicht bestätigter Einträge

```

74 def schlüsselExemplare = bundExemplar.schlüsselExemplare
75 schlüsselExemplare.each { se ->
76   // Lädt das referenzierte Schlüsselobjekt nach, damit alle benötigten Daten für den Bericht vorhanden sind.
77   def seObjekt = loadObjekt(se.objekt.id)
78   result << [
79     'schrank' : eintrag.exemplar.objekt.schrank?.anzeigename,
80     'lagerplatz' : eintrag.exemplar.objekt.lagerplatz?.name,
81     'objektid' : eintrag.exemplar.objekt.id,
82     'haken' : eintrag.exemplar.objekt.hakenAnzeige,
83     'gravur' : seObjekt.gravur,
84     'name' : seObjekt.name,
85     'nummer' : seObjekt.nummer,
86     'exemplar' : se.nummerzusatz?.toString(),
87     'objektArtText' : messages.getMessage(objektArt.SCHLUESSEL),
88     'kommentar' : eintrag.kommentar?: '',
89     'besitztyp' : eintrag.besitzer?.objektBesitzerTyp?.id?: '',
90     'besitzer' : eintrag.besitzer?.anzeigename?: '',
91     'status' : statusText,
92     'bereinigt' : InventurEintragStatus.BEREINIGT.equals(eintrag.status),
93     'type' : getType(objektArt.SCHLUESSEL, seObjekt.kritisch),
94     'schlüsselbund' : bundName,
95     'bundParent' : false,
96     'sortierNummer' : eintrag.exemplar.objekt.sortierNummer,
97     'sortierExemplar' : se.nummerzusatz
98   ]
99 }
100

```

Abbildung : Filterung der Inventureinträge für die Inventurdifferenzliste
 Abbildung : Groovy-Skript zur Filterung nicht bestätigter Einträge

Abbildung : Filterung der Inventureinträge für die Inventurdifferenzliste

Abbildung : Filterung der Inventureinträge für die Inventurdifferenzliste
 Abbildung : Groovy-Skript zur Filterung nicht bestätigter Einträge

6.3.4 Inventurdifferenzen

Abbildung 22: Erweiterung der Objekt-View um Schrank- und Gebäudedaten

```
26 def schrankView = new View(Schrank.class)
27   .addProperty("anzeigeName")
28   .addProperty("gebaeude", new View(Gebaeude.class).addProperty("name"))
29
30 def objektView = new View(Objekt.class)
31   .addProperty("nummer")
32   .addProperty("name")
33   .addProperty("gravur")
34   .addProperty("kritisch")
35   .addProperty("schrank", schrankView)
36   .addProperty("lagerplatz", new View(Lagerplatz.class).addProperty("name"))
37
```

Hinweis: Die View wurde erweitert, damit verschachtelte Gebäudedaten im Bericht korrekt geladen werden.

7 Eidesstattliche Erklärung

Hiermit erkläre ich, **Mousa Al Ataallah**, an Eides statt, dass ich die vorliegende Arbeit selbstständig und ausschließlich mit den angegebenen Hilfsmitteln und Quellen angefertigt habe. Sämtliche Stellen, die dem Wortlaut oder dem Sinn nach anderen Werken entnommen wurden, sind durch Quellenangaben kenntlich gemacht. Die Arbeit wurde weder vollständig noch in Teilen bereits anderweitig als Prüfungsleistung vorgelegt.

Mir ist bekannt, dass eine falsche Erklärung rechtliche Folgen nach sich ziehen kann.

Stuttgart 08.06.2026
Ort, Datum


Unterschrift